Drawing Picts

Name: _____ Date: _____

`Graphics2D` is a built-in Java class. `fillOval(x, y, w, h)` and `fillRect(x, y, w, h)` fill a shape whose top-left corner is at `(x, y)` (increasing y goes **down**). These are from `Pict.java`, as written (constructors left out).

```
record Circle(int radius, Color color)
    implements Pict {
  public int width() { return 2 * this.radius; }
  public int height() { return 2 * this.radius; }
  public void draw(Graphics2D g, int x, int y) {
    g.setColor(this.color);
    g.fillOval(x, y,
        2 * this.radius, 2 * this.radius);
  }
}
record Rect(int width, int height, Color color)
    implements Pict {
  public void draw(Graphics2D g, int x, int y) {
    g.setColor(this.color);
    g.fillRect(x, y, this.width, this.height);
  }
}
```

`a.overlay(b)` makes `new Overlay(a, b)`, with `a` in front, and `a.beside(b)` makes `new Beside(a, b)`. The picture this sheet draws:

```
Pict p = new Circle(10, "red")
  .overlay(new Rect(30, 30, "blue"))
  .beside(new Rect(10, 30, "green"));
```

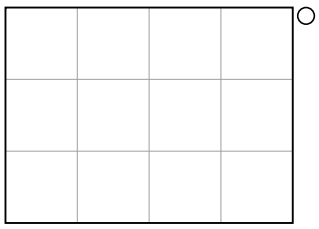
```
record Beside(Pict left, Pict right)
    implements Pict {
  // width(): the two widths added
  // height(): the larger of the two heights
  public void draw(Graphics2D g, int x, int y) {
    int centerY = this.height() / 2;
    this.left.draw(g, x,
        y + centerY - this.left.height() / 2);
    this.right.draw(g, x + this.left.width(),
        y + centerY - this.right.height() / 2);
  }
}
record Overlay(Pict front, Pict back)
    implements Pict {
  // width(): the larger of the two widths
  // height(): the larger of the two heights
  public void draw(Graphics2D g, int x, int y) {
    int cx = this.width() / 2;
    int cy = this.height() / 2;
    this.back.draw(g,
        x + cx - this.back.width() / 2,
        y + cy - this.back.height() / 2);
    this.front.draw(g,
        x + cx - this.front.width() / 2,
        y + cy - this.front.height() / 2);
  }
}
```

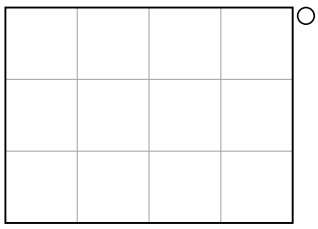
Drawing `p`

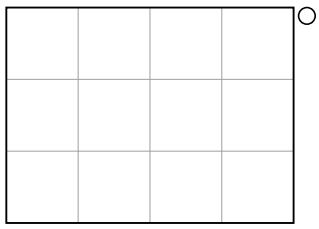
Q1. Fill in each piece's width and height.

the `Circle`: _____ by _____ the `Overlay`: _____ by _____ `p`: _____ by _____

Q2. `p.render()` starts with `p.draw(g, 0, 0)`. Fill in the calls that follow, with their arguments and the local each `draw` computes. Then, above each pair of calls on `g`, shade what the picture looks like right after them, and fill the dot with `g`'s color.







NAMES (THE STACK), OVER TIME →

`g.setColor(blue)`

`Rect.draw(g, _____, _____)`

`Circle.draw(g, _____, _____)`

`Overlay.draw(g, _____, _____)`
`cx = _____`
`cy = _____`

`Beside.draw(g, 0, 0)`
`centerY = _____`

Equal pictures

Someone adds a shape of their own, as a class:

```
class Dot implements Pict {
  int size;
  Dot(int size) { this.size = size; }
  public int width() { return size; }
  public int height() { return size; }
  public void draw(Graphics2D g, int x, int y) {
    g.setColor(Color.BLACK);
    g.fillOval(x, y, size, size);
  }
}
```

Q3. This is `false`, even though `Beside` and `Rect` are records. Why?

```
new Dot(5).beside(new Rect(3, 3, "red"))
    .equals(new Dot(5).beside(new Rect(3, 3, "red")))
```

Q4. Fill in `Dot`'s `equals` so the comparison above is `true`.

```
@Override
public boolean equals(_________________) {
  if (_____________________________) { return false; }
  Dot d = ___________________;
  return ____________________;
}
```

`instanceof` and casts

Q5. Circle each answer for `p` from the front.

`p instanceof Beside`	true false	`p instanceof Circle`	true false
`p instanceof Pict`	true false	`p instanceof Object`	true false

Q6. For each line, circle what happens. If it runs, write the value.

code	what happens	value, if it runs
`((Beside) p).left()`	runs crashes won't compile	
`((Circle) p).radius()`	runs crashes won't compile	
`Rect r = new Rect(3, 3, "red");` `Circle c = (Circle) r;`	runs crashes won't compile	

Q7. The last two both cast to `Circle`. Why does Java let one compile and not the other?